

RailKeeper / ECoS Locomotive Feldvergleich

Stand: 13.05.2026

Quelle ECoS: <https://github.com/cbries/ecos/blob/master/ECoSEntities/Locomotive.cs> Ergaenzende Quelle ECoS-Verbindung/CV-Test: <https://github.com/cbries/ecos/blob/master/ECoSConnect/Program.cs>
RailKeeper-Quelle: backend/internal/application/vehicles.go, frontend/src/shared/api.ts, Fahrzeug-Migrations

Kurzfazit

Die ECoS-Bibliothek ist fuer RailKeeper brauchbar, aber nicht als Parser fuer .eco-Backup-Dateien. Sie eignet sich als Vorlage fuer eine Live-Anbindung ueber das ECoS-Protokoll.

Bidirektional ist machbar, aber nicht fuer alle Felder gleich sinnvoll:

? Sehr gut bidirektional: Lokname, Digitaladresse, Protokoll, Funktionstasten-Metadaten nach Mapping, optional Sperr-/Sync-Status.

? Gut lesend, vorsichtig schreibend: Funktionszustaende, Fahrtrichtung, Geschwindigkeit, Fahrstufe.

? Nicht direkt ueber ECoS-Locomotive abgedeckt: Hersteller, Artikelnummer, Epoche, Spur, Kategorie, Bilder, Beilagen, Wartung, Listenpreis, EAN und weitere Inventardaten.

? Fuer RailKeeper sinnvoll zu ergaenzen: externe ECoS-ID, Digitaladresse, Digitalprotokoll, ECoS-Funktionssymbol-ID, Sync-Richtung/Sync-Status.

ECoS Locomotive Felder

ECoS-Feld	Bedeutung	In Locomotive.cs verarbeitet	Relevanz fuer RailKeeper
ObjectId	ECoS-Objekt-ID der Lok	ja, geerbt ueber Item	Muss fuer Sync persistiert werden
Name	Lokname	ja	Direkt auf RailKeeper name abbildbar
Protocol	Digitalprotokoll, z. B. MM/DCC	ja	RailKeeper braucht eigenes Feld digitalProtocol
Addr	Digitaladresse	ja	RailKeeper braucht eigenes Feld digitalAddress
Speed	aktuelle Geschwindigkeit/Fahrstufe	ja	Live-Status, nicht Inventarstamm
Speedstep	aktuelle Fahrstufe	ja	Live-Status, optional Steuer-/Testansicht
Direction	Fahrtrichtung	ja	Live-Status, optional Steuer-/Testansicht
Funcset	Funktionszustand je Funktion	ja	Live-Status; RailKeeper speichert aktuell Funktionsdefinitionen
Funcdesc	ECoS-Funktionsbeschreibung/-Symboltyp je Index	ja	Mit Mapping auf RailKeeper symbolKey/functionType nutzbar
NrOfFunctions	Anzahl Funktionen	ja, aus Funcset abgeleitet	In RailKeeper aus gespeicherten Funktionen ableitbar
Locked	Sperrstatus	ja	Neues Sync-/Sperrfeld sinnvoll
MaxSpeedFahrstufe	maximale Fahrstufe	ja	Neues optionales Digital-Tuning-Feld
BlockSpeedFahrstufe	Block-/Begrenzungsfahrstufe	ja	Neues optionales Digital-Tuning-Feld
StartTime / StopTime	Laufzeit im Client	ja, lokal berechnet	Nicht als Stammdatum geeignet
IsForward / IsBackward	abgeleitet aus Direction	ja	Nur Live-Anzeige

RailKeeper Fahrzeugfelder

RailKeeper-Feldgruppe	Beispiele	ECoS-Abdeckung	Bewertung
Identifikation	inventoryNumber, name, vehicleNumber, series	name direkt, Rest nicht	Name direkt; Rest bleibt RailKeeper
Hersteller / Artikel	manufacturer, articleNumber, articleSourceUrl, ean	nicht in Locomotive.cs	Nicht ueber ECoS-Lokmodell
Modellklassifikation	gauge, epoch, railwayCompany, category, gattung	nicht in Locomotive.cs	RailKeeper fuehrend
Digitaldaten	digital, digitalDecoderNumber, dtDecoder, dtDecoderNumber, abcBrakes	addr, protocol, teils CV-Test in Program.cs	RailKeeper erweitern statt zweckentfremden
Technik/Bauart	Laenge, Gewicht, Achsen, Kupplung, Stromabnahme, Adapter	nicht in Locomotive.cs	RailKeeper fuehrend
Licht/Sound/Rauch/Antrieb	bool + Beschreibung	teils ueber funcdesc ableitbar	Importvorschlag moeglich, keine sichere Vollautomatik
Bilder/Uploads	images, attachments, cvFiles	nicht in Locomotive.cs	RailKeeper fuehrend
Wartung	maintenance	nicht in Locomotive.cs	RailKeeper fuehrend
Funktionstasten	functionKey, name, symbolKey, functionType, mode, directionDependent, notes	funcset, funcdesc	Sehr guter Kandidat mit Mapping
CV-Werte	cvNumber, value, decoderProfile, sourceFileId	Program.cs testet get(... cv[x:y]), Locomotive.cs nicht	Separates ECoS-CV-Modul pruefen

Feldmatrix fuer bidirektionale Synchronisation

Thema	ECoS	RailKeeper heute	Richtung	Umsetzung
Externe ID	ObjectId	fehlt	ECoS -> RK	Feld/Tabelle vehicle_external_refs oder ecos_object_id
Lokname	Name	name	beide	Direkt synchronisierbar, Konfliktlogik noetig
Digitaladresse	Addr	kein klares Feld	beide, nach Erweiterung	Neues Feld digitalAddress; nicht mit Decoder-Nr. vermischen
Digitalprotokoll	Protocol	kein Feld	beide, nach Erweiterung	Neues Feld digitalProtocol mit Werten DCC/MM/SX usw.
aktuelle Geschwindigkeit	Speed	fehlt	ECoS -> RK live	Nicht als Stammdatum, sondern Live-Status
Fahrstufe	Speedstep	fehlt	ECoS -> RK live	Live-Status/Diagnose
Fahrtrichtung	Direction	fehlt	ECoS -> RK live	Live-Status/Diagnose
Funktionszustand	Funcset	keine Zustandsablage	beide live	Optional fuer Steuerungsansicht
Funktionsdefinition	Funcdesc	VehicleFunction	beide, mit Mapping	Mapping ECoS-Code <-> Symbol/Funktionstyp
Anzahl Funktionen	NrOfFunctions	aus functions ableitbar	ECoS -> RK	Beim Import Funktionen F0..Fn anlegen
Sperrstatus	Locked	fehlt	beide, nach Erweiterung	Sync-Sperre oder ECoS-Lock als separates Feld
Max-/Blockfahrstufe	MaxSpeedFahrstufe, BlockSpeedFahrstufe	fehlt	ECoS -> RK optional	Digitale Detailfelder ergaenzen
CV-Werte	nicht in Locomotive.cs, Test in Program.cs	VehicleCVValue	ECoS -> RK experimentell	Separates Modul, erst mit echter ECoS testen

Empfohlene Umsetzung in RailKeeper

- Backend-Modul ecos
 - TCP-Verbindung zu IP/Port 15471 - Request/Reply-Parser fuer ECoS-Kommandobloecke - Timeout, Fehlerstatus, Protokoll-Logging
- Datenmodell erweitern

- vehicles.digital_address - vehicles.digital_protocol - vehicle_external_refs fuer ecos:ObjectId - optional vehicle_ecos_sync fuer Richtung, Status, letzte Synchronisation

3. Import-Assistent

- Verbindung testen - ECoS-Lokliste laden - RailKeeper-Matching nach ECoS-ID, Name, Digitaladresse - Vorschau: neu, geaendert, Konflikt, ignoriert - bewusste Uebernahme je Feld

4. Export/Synchronisation zur ECoS

- Start mit sicherem Schreibumfang: Name, ggf. Funktionstasten-Mapping - Adresse/Protokoll erst nach realem Geraetetest freigeben - Geschwindigkeit/Richtung/Funktionen nur in Live-Steuerung, nicht als Massenexport

5. Funktionstasten maximal nutzen

- ECoS funcdesc mit vorhandenen RailKeeper-Symbolen mappen - unbekannte ECoS-Codes als neue Symbolzuordnung sichtbar machen - Richtung/Modus in RailKeeper erhalten, ECoS nur soweit das Protokoll es traegt

6. Release fuer Test

- Erstes Release sollte die aktuelle stabile App plus die Vorbereitungen enthalten - ECoS-Anbindung danach als experimenteller Bereich hinter Feature-Schalter - Fuer echte Tests braucht es IP-Adresse der ECoS und mindestens eine Testlok mit Funktionen

Risiko- und Testpunkte

? ECoS-Live-Protokoll ist nicht dasselbe wie .eco-Backup-Datei.

? Schreibende Syncs koennen Daten auf der Zentrale veraendern; deshalb zuerst Preview und Feldfreigabe.

? Addr ist die Digitaladresse, nicht zwingend die Decoder-Seriennummer.

? Funcdesc ist ohne Mapping nur ein numerischer Code; RailKeeper braucht eine stabile Symbol-Mapping-Tabelle.

? CV-Zugriff ist im Repo nur als Testkommando sichtbar, nicht als fertiges Objektmodell.

Erste sinnvolle Release-Stufe

Release 0.1.x sollte enthalten:

? aktueller RailKeeper-Stand mit Benutzerverwaltung, Messeliste, CV-/Funktionssymbolbasis und i18n-Fixes

? PDF-Feldvergleich als Dokumentation

? offener, dokumentierter Punkt "ECoS Live-Sync"

Naechster Entwicklungsblock:

? ECoS-Verbindungstest im Backend

? API GET /api/v1/ecos/status

? API POST /api/v1/ecos/locomotives/preview

? UI im Import/Export-Bereich als experimenteller ECoS-Import